Modelplane

SOVEREIGN CLOUD DAYS 2026 · WORKSHOP

From model to sovereign endpoint

A Modelplane walkthrough, live on two providers

Christopher Haar · Yury Tsarev · Saarbrücken · 24 October 2026

Modelplane

SPEAKERS

Who's driving today



Christopher Haar

Staff Software Engineer

Upbound

A core maintainer and regular contributor in the Crossplane ecosystem. Enterprise infrastructure across finance, telco, railway and cloud.

crossplane · modelplane · provider-aws · DevEx · platform-reference · provider-castai · x-metrics · provider-pagerduty · provider-zscaler · SDLC · x-generate · Metrics Operator · solutions · CLI



Yury Tsarev

Principal Solutions Architect

Upbound

Helps some of the world's largest enterprises build evolvable infrastructure with Crossplane and scale it with Upbound. Open source contributor to the Crossplane ecosystem (CNCF Graduated), creator of k8gb (CNCF Incubating).

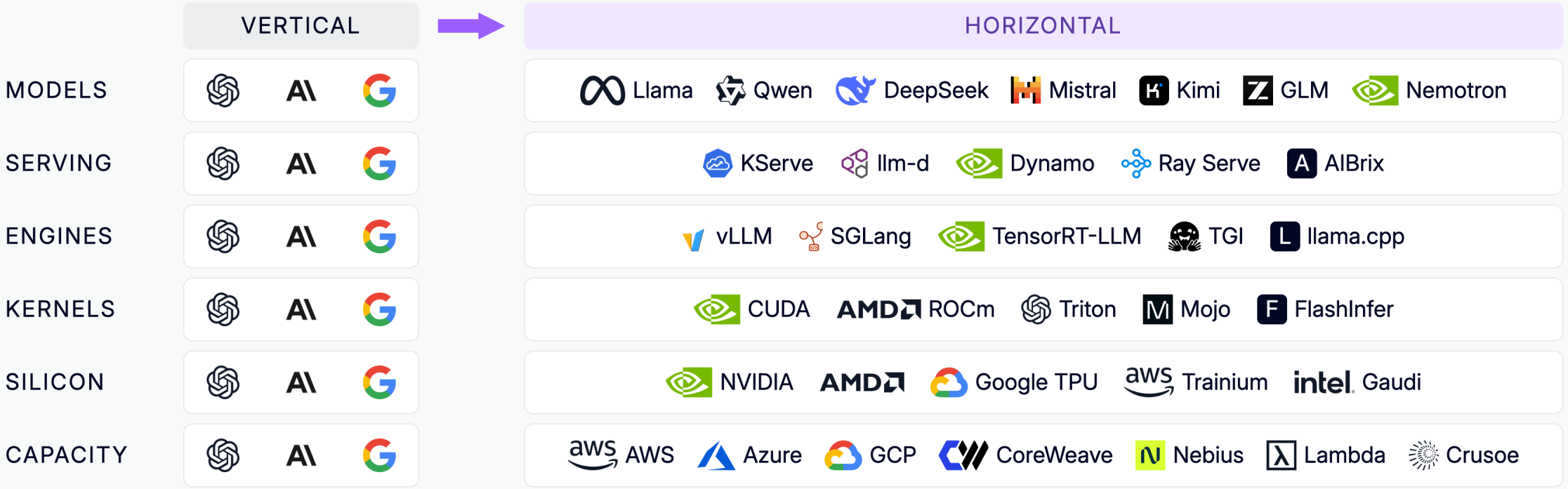
crossplane · modelplane · k8gb · function-azresourcegraph · function-msgraph · function-aws-query · test-driven infrastructure · Upbound reference platforms

Modelplane

WHY

The inference market is going horizontal

Vertical: one vendor decides every layer, and where it runs. Horizontal: you choose each one.



Modelplane

THE PROBLEM

Before you get a single token back

Four layers, each with good open options, and none of them knows about the others.

- x **Serving engine** — vLLM · SGLang · TensorRT-LLM · Flags · KV cache · Experts
- x **GPU scheduling** — DRA · VRAM · Gang scheduling · Multi-node
- x **Model storage** — Weights · Hugging Face · OCI · ModelPack
- x **Routing** — Endpoints · OpenAI-compatible · Anthropic-compatible · Replicas

No shared context between the layers

Modelplane

THE PROBLEM

Inference doesn’t fit on one cluster

Serving a model on one cluster is solved. Serving it across many isn’t.

- **GPUs are scarce and scattered**
  A cloud region, a neocloud, a rack you own
- **Capacity is unevenly free**
  The model has to land where the right GPU is idle today
- **Every environment differs**
  Accelerators, fabrics, quotas and compliance zones
- **Teams want one endpoint**
  Not one endpoint per cluster

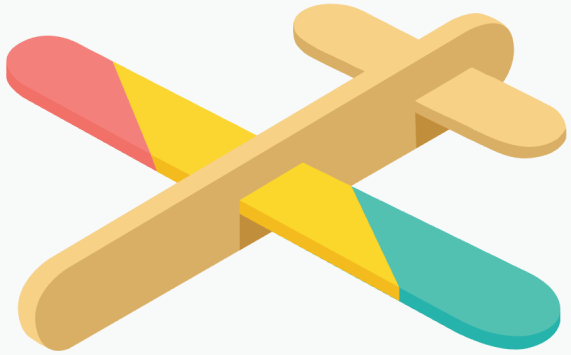
THE GAP

No open system manages an inference fleet as a whole: placing deployments across clusters, scaling replicas, moving weights, routing to whichever cluster is serving.

Kubernetes became the control plane for compute. Crossplane extended it to cloud infrastructure. Inference needs the same layer.

INTRODUCTION

The open source control plane for AI inference



Modelplane

provisions · schedules · caches · routes · scales · observes

Built on Crossplane, a CNCF graduated project · Apache 2.0 · v0.5, five releases since 23 June 2026

MODELPLANE

What it is, and what it isn't

Open source, Apache 2.0, built on Crossplane, built in the open.

Is

- A control plane you install and run, on a cluster of your own
- It builds clusters, places replicas where they fit, caches weights, routes traffic and collects metrics
- Any container engine, any accelerator: cloud, neocloud, on-prem, or all at once
- Always reconciling the fleet toward the state you declared

Is not

- A serving engine: vLLM, SGLang and TensorRT-LLM stay as they are, and Modelplane composes them
- A managed service: it's software you run, not an API you rent
- Opinionated about engine flags: parallelism, quantization and KV transfer are yours to write, and Modelplane never injects them

WHY

Sovereign means you decide where it runs

What	Where it lives with Modelplane	How
Control plane	Where you run it: today, on this laptop	Crossplane
Placement	Only on clusters your labels and policy allow	clusterSelector
Weights	Staged once, into storage you own	ModelCache
Requests	Through a gateway in your region, with your keys	InferenceGateway
Records	Usage, metrics and audit, in your backend	TelemetryDestination

Each row is a stop in the walkthrough that follows.

Modelplane

Demo: ./demo.sh dra

Before DRA, a GPU is a label and a count

The node carries a label

```
$ kubectl get nodes -L nvidia.com/gpu.product
NAME                                GPU.PRODUCT
aks-gpua100-31415926-vmss000000     NVIDIA-A100-80GB-PCIe
```

The pod asks for a count

```
resources:
  limits:
    nvidia.com/gpu: 1
```

Labels steer: a nodeSelector matches the product name.

The request is an integer: one GPU, any GPU on that node.

No way to ask for “at least 14 GiB” or “compute capability 8.0 or newer”.

Modelplane

Demo: ./demo.sh dra

With DRA, the driver publishes every device

The A100 node's ResourceSlice, abridged

```
kind: ResourceSlice
spec:
  driver: gpu.nvidia.com
  nodeName: aks-gpua100-31415926-vmss000000
  devices:
  - name: gpu-0
    attributes:
      productName: {string: NVIDIA A100 80GB PCIe}
      architecture: {string: Ampere}
      cudaComputeCapability: {version: 8.0.0}
    capacity:
      memory: {value: 80Gi}
```

One ResourceSlice per node and driver, kept current by the driver.

Typed attributes: strings, versions, booleans. Capacity as quantities.

Pods stop counting GPUs. They claim devices that match.

Modelplane

Demo: ./demo.sh dra

Claim a device by what it is, in CEL

What the engine pod claims

```
kind: ResourceClaimTemplate
spec:
  spec:
    devices:
      requests:
      - name: gpu
        exactly:
          deviceClassName: gpu.nvidia.com
          count: 1
          selectors:
          - cel:
              expression: >-
                device.capacity["gpu.nvidia.com"].memory
                  .compareTo(quantity("14Gi")) >= 0
```

The DeviceClass names the driver. CEL filters its devices.

The scheduler binds a device that passes, or the pod waits.

Ask for usable memory, not the box label: an 80GB H100 reports about 81559Mi.

Modelplane checks the same CEL one level up

```
device.capacity["gpu.nvidia.com"].memory.compareTo(quantity("14Gi")) ≥ 0
```

1 · Fleet: Modelplane

Checks it against each InferenceClass, the devices a node pool declares

→ picks a cluster and a pool

2 · Node: DRA

Checks it against the real ResourceSlices on that cluster

→ binds a GPU to the pod

So a class declares what the driver will report. Declare more, and a replica lands where its GPU can't bind.

MODELPLANE

Eight objects, two teams

Platform describes the fleet, ML describes a model, Modelplane composes the rest.

PLATFORM TEAM WRITES	ML TEAM WRITES	MODELPLANE WRITES
`InferenceClass` A node pool's hardware, as DRA reports it, and how to build it	**`ModelCache`** Weights staged once per cluster, on shared storage	*You never touch these directly. Modelplane keeps them in step.*
`InferenceCluster` A cluster in the fleet: built by Modelplane, or yours, registered	**`ModelDeployment`** The model, engine and replicas: what it needs, not where	**`ModelReplica`** One serving instance, pinned to a cluster and a pool
`InferenceGateway` The address callers use: OpenAI and Anthropic APIs	**`ModelService`** One model name for callers, across every replica	**`ModelEndpoint`** One per replica, or written by hand for an external provider

MODELPLANE

If you know Kubernetes, you know this

In one cluster	Across the fleet	What it is
Deployment	ModelDeployment	How many, of what
Pod	ModelReplica	One running instance, on a chosen cluster
Service	ModelService	A stable name in front of many instances
Endpoints	ModelEndpoint	The backends a request can reach

One scope up. Modelplane writes the replicas and endpoints; you read them to see where things landed.

WALKTHROUGH · TODAY'S SETUP

One control plane, above the fleet

Modelplane installs the serving stack on each cluster it builds, or brings in one you already run.

MODELDEPLOYMENT
ml-team/qwen-7b

vLLM · Qwen2.5-7B AWQ · 1× GPU ≥ 14 GiB · replicas: 1 → 2 · jurisdiction: eu

Modelplane — kind, on this laptop

PROVISIONING · SCHEDULING · CACHING · ROUTING · SCALING · OBSERVING

POLICY · RBAC · GITOPS *from your stack, around it*

INFERENCECLUSTER
AKS
Cloud · swedencentral
1× A100 80GB · gateway

INFERENCECLUSTER
Nebius
Neocloud · eu-north1
1× L40S 48GB

INFERENCECLUSTER
Your own
On-prem · source: Existing
Not in today's demo

PLATFORM TEAM · 1 OF 3

Demo: ./demo.sh fleet

InferenceClass

A node pool's hardware recipe: how to build it, and what it offers.

```yaml
kind: InferenceClass
metadata: {name: a100-1x}
spec:
  provisioning:          # 1 how to build one
    provider: AKS
    aks:
      vmSize: Standard_NC24ads_A100_v4
      accelerator: {type: nvidia-a100, count: 1}
  devices:               # 2 what it offers
  - name: gpu
    claim: DRA
    driver: gpu.nvidia.com
    attributes:
      architecture: {string: Ampere}
      cudaComputeCapability: {version: "8.0.0"}
    capacity:
      memory: {value: "81920Mi"}  # 3
```

1. **How to build one**
   Cloud-specific: VM size and accelerator, used when Modelplane builds the pool

2. **What it offers**
   Cloud-neutral: the devices a deployment's CEL is matched against

3. **Usable, not nominal**
   The A100 reports 80Gi. An L40S reports 46068Mi, not 48GB.

Modelplane

PLATFORM TEAM · 2 OF 3

Demo: ./demo.sh fleet

Stack from NVIDIA AICR recipes

InferenceCluster

One object becomes a GPU cluster, with its serving stack.

The platform team writes

```
apiVersion: modelplane.ai/v1alpha1
kind: InferenceCluster
metadata:
  name: aks-swedencentral
  labels:
    jurisdiction: eu
spec:
  cluster:
    source: AKS
    aks: {location: swedencentral}
  nodePools:
  - name: gpua100
    className: a100-1x
    nodeCount: 1
```

15 Helm releases: GPU operator, NVIDIA DRA driver, Envoy and AI Gateway, LeaderWorkerSet, Prometheus

24 more Kubernetes objects, installed in dependency order

~20 minutes from apply to Ready on AKS

Modelplane

PLATFORM TEAM · 3 OF 3

▷ Demo: ./demo.sh fleet

InferenceGateway

The front door: one address, the OpenAI API at /v1 and Anthropic's at /anthropic/v1.

```
kind: InferenceGateway
metadata: {name: eu}
spec:
  clusterName: aks-swedencentral    # 1
  auth:                             # 2
    method: APIKey
    apiKey:
      secretSelector:
        matchLabels:
          modelplane.ai/inference-keys: "true"
  serviceSelector:                  # 3
    matchLabels: {jurisdiction: eu}

$ kubectl get inferencegateway
NAME   CLUSTER             ADDRESS
eu     aks-swedencentral   203.0.113.24
```

1. **Runs on a cluster you name**
   One per cluster. Put one wherever requests should enter.

2. **Keys you hold**
   API keys in Secrets on your control plane, picked by label

3. **Residency by label**
   EU services reach only EU gateways, then only their own endpoints

Modelplane

PLATFORM TEAM

Demo: ./demo.sh fleet

The platform team is done

```
$ kubectl get inferenceclusters
NAME                SOURCE   GATEWAY           SYNCED   READY   AGE
aks-swedencentral   AKS      203.0.113.10      True     True    2d4h
nebius-eu-north     Nebius   198.51.100.7      True     True    1d22h

$ kubectl get inferencegateways
NAME   CLUSTER             ADDRESS
eu     aks-swedencentral   203.0.113.24
```

Capacity is published once. From here, ML teams self-serve.

Modelplane

ML TEAM · 1 OF 5

▷ Demo: ./demo.sh cache

ModelCache

Weights are staged once per cluster, inside your boundary.

The ML team writes

```
apiVersion: modelplane.ai/v1alpha1
kind: ModelCache
metadata:
  name: qwen-7b
  namespace: ml-team
spec:
  source: HuggingFace
  huggingFace:
    repo: Qwen/Qwen2.5-7B-Instruct-AWQ
    sizeGiB: 10
  clusterSelector:
    matchLabels:
      jurisdiction: eu
```

5.6 GB of weights, staged once per EU cluster on shared storage

~2 min from zero replicas back to serving, from the warm cache

2 copies, one per EU cluster, and none anywhere else

ML TEAM · 2 OF 5

Demo: ./demo.sh govern

ModelDeployment

Declare what the model needs, not where it runs.

```
kind: ModelDeployment
metadata: {name: qwen-7b, namespace: ml-team}
spec:
  replicas: 1
  template:
    spec:
      clusterSelector:                # 1
        matchLabels: {jurisdiction: eu}
      modelCacheRef: {name: qwen-7b}  # 4
      engines:
      - name: qwen-7b
        members:
        - role: Standalone
          nodeSelector:               # 2
            devices: [{name: gpu, count: 1, selectors: […]}]
          template:                   # 3
            spec:
              containers: [{image: vllm/vllm-openai:v0.9.2}]
```

1. **Where it may run**
   Clusters labelled jurisdiction: eu
2. **What it needs**
   Any GPU with at least 14 GiB, in CEL
3. **The engine is yours**
   Image and flags, as you write them
4. **Start warm**
   Weights from the ModelCache

ML TEAM · 3 OF 5 · POLICY

▷ Demo: ./demo.sh govern

Placement you can prove

The platform team makes the EU selector mandatory, in core Kubernetes.

The rule

```
kind: ValidatingAdmissionPolicy
metadata: {name: eu-only-model-deployments}
spec:
  validations:
  - expression: >-
      has(…clusterSelector.matchLabels) &&
      …matchLabels['jurisdiction'] == 'eu'
---
kind: ValidatingAdmissionPolicyBinding
spec:
  validationActions: [Deny]
  matchResources:
    namespaceSelector:
      matchLabels: {jurisdiction: eu}
```

A deployment without the selector

```
$ kubectl apply --dry-run=server \
    -f policies/test/qwen-anywhere.yaml

denied by ValidatingAdmissionPolicy
eu-only-model-deployments:
ModelDeployments in this namespace
must set spec.template.spec
.clusterSelector.matchLabels
.jurisdiction: eu.
```

Nothing is stored. Kyverno or OPA work the same way.

ML TEAM · 4 OF 5

Demo: ./demo.sh scale

Scale out, and see where it lands

A real scale subresource. Each replica is a whole serving instance, placed on its own.

```
$ kubectl scale modeldeployment/qwen-7b \
    -n ml-team --replicas=2
modeldeployment.modelplane.ai/qwen-7b scaled

$ kubectl get modelreplica -n ml-team
NAME            CLUSTER              READY   AGE
qwen-7b-x8k2p   aks-swedencentral    True    1d21h
qwen-7b-m4q7d   nebius-eu-north      True    4m
```

Autoscale on external metrics. HPA on pod metrics can't see pods that run on other clusters.

1. **Filter clusters**
   The clusterSelector against each InferenceCluster's labels

2. **Filter pools**
   The CEL against each InferenceClass, with room for a whole replica

3. **Spread, then pin**
   To the cluster with the fewest. The first replica stays put.

ML TEAM · 5 OF 5

Demo: ./demo.sh call

ModelService

One name for callers, across every replica, wherever it runs.

```
kind: ModelService
metadata:
  name: qwen-7b
  namespace: ml-team
  labels: {jurisdiction: eu}        # EU gateways only
spec:
  endpoints:
  - name: qwen-7b
    selector:
      matchLabels:
        modelplane.ai/deployment: qwen-7b  # 1

$ kubectl get modelservice -n ml-team
NAME      MODEL
qwen-7b   ml-team/qwen-7b                  # 2
```

1. **Selects endpoints, not deployments**
   One ModelEndpoint per replica, on any cluster, labelled with its deployment

2. **One name: ml-team/qwen-7b**
   Callers send it as the model. The gateway rewrites it per backend.

3. **Weight and priority**
   Shift traffic for a rollout, or set a failover order

WALKTHROUGH · THE ENDPOINT

▷ Demo: ./demo.sh call · resilience

One endpoint, your keys, either cloud

A caller, through the gateway

```
$ curl $GATEWAY/v1/models
→ 401: no key, no answer

$ curl $GATEWAY/v1/models \
    -H "Authorization: Bearer sk-…"
→ ml-team/qwen-7b

$ curl $GATEWAY/v1/chat/completions \
    -H "Authorization: Bearer sk-…" \
    -d '{"model": "ml-team/qwen-7b", "stream": true, …}'
→ tokens, streamed
```

The gateway runs on an EU cluster and speaks the OpenAI and Anthropic APIs.

Replicas on AKS and Nebius answer to one name. Callers never see which.

A 429 or 503 is retried up to 3 times. A replica that fails 5 times in a row is ejected for 30 s.

WALKTHROUGH · DAY 2

▷ Demo: ./demo.sh lifecycle · oversight

Day 2 is an edit to an object

Change	The edit	What guards it
New weights or engine	A second deployment, then shift the weights 90/10	Callers keep one name
Cluster maintenance	A taint moves the replicas off the cluster	The other cloud keeps serving
Retire a cluster	Delete the InferenceCluster	Refused while in use, with the reason
Who may change it	Kubernetes RBAC on the objects	Only the GitOps pipeline edits

Every change goes through Git and the API, so every change has an author.

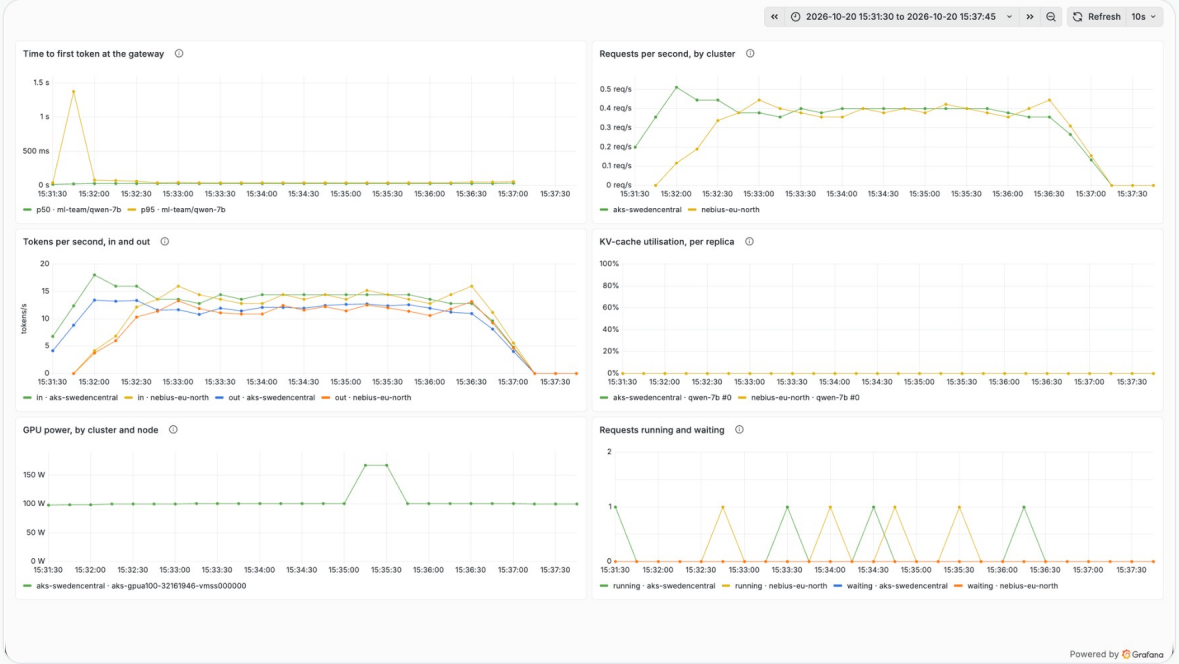
Modelplane

MONITORING

Demo: ./demo.sh monitor

Every request leaves a record, in your backend

A collector on every cluster → `TelemetryDestination` Prometheus remote write → Your Prometheus and Grafana



Time to first token and tokens, per deployment and replica

KV-cache use and GPU power, per cluster and node

Usage per caller, from the gateway

The demo's run: requests split between AKS and Nebius. Modelplane ships no dashboards; it exports modelplane_* metrics. Ours is in the demo repository.

MONITORING · METRICMAPPING

One set of metric names, whatever the engine

Written once on the control plane. Every cluster's collector applies it.

```
kind: MetricMapping
metadata: {name: my-engine}
spec:
  metrics:
  - from: my_engine_queued_requests          # 1
    to: modelplane_requests_waiting
  - from: my_engine_kv_transfer_ms           # 2
    fromUnit: Milliseconds
    to: modelplane_request_kv_transfer_seconds
  - from: my_engine_prompt_tokens_total      # 3
    to: modelplane_tokens_total
    labels:
    - {name: direction, value: input}
```

1. **Your engine's names, the fleet's names**
   vLLM, SGLang, GPUs and the gateway come mapped. This is for anything else.

2. **Units converted, not trusted**
   A _seconds series holding milliseconds would read a thousand times fast.

3. **Fold metrics into one name**
   A fixed label tells input tokens from output tokens.

Only modelplane_* leaves a cluster. Anything no mapping renames stays where it was scraped.

HONEST LIMITS · V0.5

The hard parts

Each open item is a decision or a known gap, and none of them is a surprise.

GPU scheduling

TODAY

- Two levels: Modelplane picks cluster and pool, DRA binds the GPU
- Spreads before packing; a multi-node replica lands whole

OPEN

- A whole node is charged per pod, so GPUs can strand (#172)
- No re-placement off a degraded cluster yet (#105)

Model caching

TODAY

- Fetched once per cluster, only where its selector allows
- Replicas mount it; scale to zero and it stays warm

OPEN

- The engine still calls Hugging Face at startup (#530)
- Big models can load slower off shared storage (#204)

Routing

TODAY

- A gateway per region, mutual TLS to every cluster
- In-cluster routing by prefix cache and load

OPEN

- Data-parallel attention loses prefix-cache reuse (#500)
- SGLang prefill/decode doesn't transfer KV yet (#502)

Also not yet: dashboards and alerts · API keys scoped to some models · a settled API (v1alpha1)

PROJECT · V0.5

Where it is

v1alpha1

The API still moves: the deployment spec moved under template in v0.2, and the gateway became an AI gateway in v0.5.

Apache 2.0

Open from the first commit, and intended for a neutral open source foundation later this year.

6 providers

AKS, EKS, GKE, Nebius, Vultr and Civo, plus any conformant Kubernetes cluster you register.

9 recipes

Worked, copyable manifests for real models, from Qwen2.5-7B to Kimi K2, instead of engine support hard-coded in the API.

Modelplane

RELEASES

Where it’s been, and what’s next

A release a month since 23 June 2026, and all five have shipped.

23 Jun

v0.1

Provisioning, two-level scheduling, caching, one OpenAI gateway

EKS · GKE

16 Jul

v0.2

Weighted traffic splits, endpoint picker, taints for drains

+ Nebius · AKS

12 Aug

v0.3

Anthropic API, model recipes, a local two-cluster e2e test

+ Vultr

10 Sep

v0.4

A serving stack per cluster: Standard or NVIDIA Dynamo

+ Dynamo stack

2 Oct · today

v0.5

AI gateway, fleet metrics, scale to zero, deletion guard

+ Civo

NEXT: OPEN IN THE TRACKER, NOT DATED

Models from OCI registries (#512) · Start from the cache without Hugging Face (#530) · Gateway usage and per-deployment metrics (#534, #535) · Re-placement off a degraded cluster (#105)

Modelplane

Your model, your clusters, your endpoint

CODE	github.com/modelplaneai/modelplane
DOCS	docs.modelplane.ai/getting-started
TODAY'S DEMO	github.com/upbound/modelplane-sovereign-demo

Open source, Apache 2.0. Questions now, or find us after the session.